AUTODEVELOPMENT 落地 · 进展与后续计划

按 10.25 董事长交流修正打法：**小步快跑、急用先行**，先把 MVP 做出来，再逐步扩充应用场景。

1. 已商定以 EIS 为试验局点，争取**一周内**跑通首个端到端（从需求到产品）研发流程，看到效果
2. 按照 Anthropic 人与 AI 协同的工程架构进行复制
3. 黑灯工厂：研发流程 24 小时运转，研发效能的度量从面向人的 “**X 人团队、XX 人月工作量**” 到面向 AI 的 “**X 个 Agent 实例、消耗 XX Tokens**”

落地计划 · 2026 年 11–12 月

#	执行事项	时间节点	说明
1	**以 EIS 为实验局，跑通 AutoD 流水线**	**11.2–11.30**	小步快跑、急用先行
1.1	搭建 AutoD 流水线，部署 EIS 产品为试点	**11.2**	**流水线 MVP 版搭建已完成**，EIS 10.30 后投入
1.2	E2E 跑通首个 EIS 纯后端需求	**11.9**	1 周
1.3	扩展至带前端、跨服务等全量场景	**11.16**	1 周
1.4	人与 AI 对抗，充分验证可行性后，全面接入 EIS 研发生产环境	**11.30**	2 周
2	**推动公司其他软件平台（Avant Hub、SkyShield）导入 AutoD 流水线**	**12月**	复制 EIS 试点经验
3	**启动其他领域数字员工建设**	**12月内**	复制 Anthropic 协同架构

集团研发效能 · AI 研发体系专题

Anthropic 工程方法
AI-人类协同开发的工程架构

当 AI 接管代码执行，工程师的角色、协作方式与管理控制点都需要重新设计。 这套架构一句话概括：人负责意图与裁决，AI 负责执行，驾驭层（Harness）把人的判断固化成系统约束。

工程方法 · 引用来源

01 **Running an AI-native engineering org**
https://claude.com/blog/running-an-ai-native-engineering-org

02 **Context engineering cookbook**
https://platform.claude.com/cookbook/tool-use-context-engineering-context-engineering-tools

03 **The AI-native SDLC playbook**
https://academy.claude.com/courses/ai-native-sdlc-playbook

04 **How Claude Code works**
https://code.claude.com/docs/en/how-claude-code-works

05 **Effective Context Engineering for AI Agents**
https://anthropic.com/engineering/effective-context-engineering-for-ai-agents

06 **Effective Harnesses for Long-Running Agents**
https://anthropic.com/engineering/effective-harnesses-for-long-running-agents

07 **Equipping Agents with Agent Skills**
https://anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

08 **Steering Claude Code: skills, hooks, subagents and more**
https://claude.com/blog/steering-claude-code-skills-hooks-rules-subagents-and-more

09 **Agent Harness Design: 3 Patterns for Harnessing Claude's Intelligence**
https://claude.com/blog/harnessing-claudes-intelligence

10 **Claude Agent SDK Overview**
https://code.claude.com/docs/en/agent-sdk/overview

Anthropic Engineering Method: Human-AI Collaboration Architecture

Human intent, Claude execution, and Harness constraints

Human Intent Layer

Product / Business
Goals, priorities, success criteria, eval criteria

Architecture / PM
ADRs, boundaries, risk tiers, trade-offs

Human Reviewer
Security, privacy, compliance, major design, release

Ops / SRE
SLOs, rollback, production gates

Harness Layer

CLAUDE.md / AGENTS.md
rutes, boundaries, commands, style

Permissions
allow / ask / dany

Hooks
pre-commit checks, secret scanning, dangerous commands

Skills
repeatable workflows: release, migration, review

MCP
git, issue tracker, read-only DB, logs

CI / Evals / Coverage Gate
machine gates

Orchestrator + Agentic Loop

Context Claude
Plan, route, merge, keep small context

Context
read files, search code, git history, tools

Action
edit code, run command, call tool

Observe
stdout, stdermit, results, iint, type errors

Verify
did it meet the goal?
did it break constraints?

Repeat until done

Oubagents + Long-running Memory

Explore Agent	Plan Agent	Test Agent	Review Agent	Reforcor Agent	Doc Agent
search code, read architecture, find call chains	architecture draft, trade-offs, ADR draft	unit tests, coverage, edge cases	bugs, style, security, potential issues?	limited-scope rename, abstraction, cleanup	README, APIo doc, changolog

Orchestrator Claude

summary + evidence location

Feature List

claude-progress.txt
what wescription, what failed, open true/tstep.

name, drogress.txt
passes: opee: false
passes: true/false

git history
external memory

NOTES.md / MEMORY.md
cross-session state

Execution Environment

Repo + Filesystem → Shell / Container / Worturee → Preview / Staging / Prod

Human review only at high-risk decisions

Core principle: AI is the fast executor, humans are the slow but accountable descision makers, and harness turns human judgment into system constraints



第 3 页全景图 · 本页位置

第①层 / HUMAN INTENT LAYER · 人的意图层

人只输入两样东西：说清"要什么"，把住结果"对不对"。

产品 / 业务

目标

目标、优先级、成功标准、验收标准

决定 AI 做出来的东西**"对不对"**

架构 / PM

边界

架构决策记录（ADR）、系统边界、风险分级、权衡取舍

决定 AI**"按哪条路走"**，把代价高的弯路提前拦下

人类评审

担负责任

安全、隐私、合规、重大设计、发布批准

高风险节点上**说"不"**的权力，不可委托给 AI

运维 / SRE

问题兜底

SLO、回滚预案、生产环境门禁

线上事故的**最后防线**，定义"什么状态必须回滚"

管理含义：意图层的产出（目标、边界、风险分级）是后续全部自动化环节的"**宪法**"。 写不清楚的意图，AI 会替你脑补，错误往往在最后一公里才暴露，返工成本最高。 **组织要做的是把"定义与裁决"变成显式岗位产出物，而不是靠开会口头对齐。**



第 3 页全景图 · 本页位置

第②层 / HARNESS LAYER · 驾驭层

不止是 AI 与人交互的管道，还把人的判断从"语言交流"变成"系统性约束"

CLAUDE.md / AGENTS.md 员工手册

项目规则、边界、常用命令、代码风格，AI 每次开工自动读取，新会话即刻生效

Permissions 授权矩阵

allow / ask / deny 三级：什么直接做、什么先请示、什么禁止。权限即流程

Hooks 内控自动化

提交前检查、密钥扫描、危险命令拦截，机器执法、不徇私情、全量留痕

Skills 标准作业程序

发布、迁移、评审等可复用工作流，一次沉淀、全员复用，好经验不再随人流失

MCP 统一工具网关

代码库、工单、只读数据库、日志的标准接口：接 N 个工具，AI 只学一套协议

CI / Evals / 覆盖率 质检门禁

测试、评估、覆盖率由机器把守，不达标，AI 无法"宣布完成"

> **管理含义：**这层就是 AWS 文章所说的"**有约束的自由**"，工具与模型向团队开放（创新空间），用四个支柱收拢秩序：**标准接口 · 资产可见 · 架构统一 · 可维护**。制度写成文件与配置，而不是写进会议纪要，这是 Anthropic 与"用了 AI 但没章法"团队的本质区别。



第 3 页全景图 · 本页位置

第③层 / ORCHESTRATOR + AGENTIC LOOP · 执行循环

人在环外，AI 在环内：取上下文 → 动手 → 看结果 → 自查，循环直至达标。

1 CONTEXT

取上下文

读文件、搜代码、查 git 历史、调工具。先搞清现状再动手

→

2 ACTION

动手执行

改代码、跑命令、调用工具。在授权与门禁允许的范围内操作

→

3 OBSERVE

观察结果

命令输出、lint、类型错误。机器反馈直接回到 AI 手里

→

4 VERIFY

自查验收

达到目标了吗？违反约束了吗？。像验收人一样端到端验证

↻ **未达标 → 带着问题回到第 1 步**，循环直至完成（Repeat until done）；整个循环不需要人在线

官方最佳实践的核心：给 AI 配"自查手段"

让 AI 像真实用户一样用浏览器端到端验证功能，而不是"改完代码就算做完"——验证工具的有无，直接决定产出质量

Anthropic 实测最大教训

没有 Verify 环节，AI 会**提前宣布胜利**：功能没通过也标记"完成"。配了自查手段，AI 才没法"宣布虚假的完成"

管理含义

把"质量意识"内置进执行循环，而不是靠事后人工抽检；人的时间只花在**定目标**和**看最终产出**上



第 3 页全景图 · 本页位置

第④层 / SUBAGENTS · 智能体分工

不是"一个更强的 AI"，而是"一支会分工的 AI 团队"。

指挥官 **Orchestrator Claude** 拆解任务、分派工种、汇总结果——只收摘要与证据位置，不被细节淹没

探索 EXPLORE
读代码、理清架构、找调用链

计划 PLAN
架构草案、ADR 草稿、权衡清单

实现 IMPLEMENT
按计划写代码、改配置、调接口

测试 TEST
单元测试、覆盖率、边界用例

评审 REVIEW
bug、风格、安全、潜在隐患

重构 REFACTOR
小范围重构、抽象、清理

文档 DOC
README、API 文档、变更日志

+90.2% Anthropic 多智能体研究系统的内部评估：orchestrator-worker（指挥官-工人）模式比单体智能体高 **90.2%**。 分工成败取决于**委派质量**，目标、输出格式、可用工具、任务边界，**四要素缺一就会重复劳动或漏活**（实测：模糊指令会让多个子智能体做同一件事）。

管理含义：上下文隔离是刻意设计，各干各的互不干扰，指挥官只看汇总。 **"委派四要素"就是给 AI 团队的岗位说明书模板**：组织设计原理在智能体身上同样成立，写不清岗位说明书的团队，AI 也带不好。



第 3 页全景图 · 本页位置

第⑤层 / EXECUTION ENVIRONMENT / 执行环境

AI 永远在沙箱里干活；越靠近生产，人的介入越集中在高风险决策点。

试验区

REPO + FILESYSTEM

代码库与文件系统

AI 的全部工作对象都在仓库内：改动可追踪、可回滚、可评审

破坏力：**零**。改错了大不了回滚

→

隔离区

SHELL / CONTAINER / WORKTREE

容器与隔离工作树

试错被限制在容器内；多任务各用各的工作树，互不踩脚

破坏力：**有界**。爆炸半径 = 一个容器

→

生产区

PREVIEW / STAGING / PROD

预发与生产

发布必须过门禁：数据变更、权限变更、上线动作

破坏力：**真实**。人守在这里，逐项放行

管理含义：风险控制不靠"相信 AI 守规矩"，靠**环境分层**——破坏力被容器和门禁逐层截留。 人不是不在环上，而是**只在环上的关键点位**：高风险决策。这同时就是问责链的落点——出问题时，能说清是谁批的、批了什么。